The Transported Probability Density Function (PDF) Method and its Relation to the Black-Scholes-Merton Equation

Christian DiPietrantonio

ME 5311: Computational Methods to Viscous Flows
University of Connecticut
May 3, 2026

CONTENTS

I. Introduction

This paper explores the mathematical connection between the Black-Scholes-Merton (BSM) equation for pricing financial derivatives and the transported probability density function (PDF) method employed in computational fluid dynamics (CFD). The central argument is that these two equations, from completely different fields, are derived from stochastic differential equations (SDEs) of the same dift plus diffusion form, and can be solved using the same classes of numerical methods. The BSM equation is derived from the geometric brownian motion SDE, while the transported PDF equation is derived from the generalized Langevin SDE for the velocity of a fluid particle in turbulence. Even though the two equations were developed in separate disciplines, these two equations share a mathematical connection through stochastic calculus.

The paper begins by establishing the financial definitions and background necessary to the understanding of the further sections. Following this, the BSM equatin is derived following the approach outlined by Hull [1] , Black and Scholes [2], and Merton [3]. The transporded PDF method is then introduced following the work of Pope [4], and the connection between it and the BSM equation is illustrated through comparison of the underlyinbg SDEs and variable mapping. The paper then outlines the solution methods for the BSM and the transported PDF equation, which include grid-based Eulerian and particle-based Lagrangian approaches. To demonstrate these solution methods, a Python program was developed that implements three different solution methods for pricing a European call option. The first solution method is simply the analytical solution to the BSM equation. Following this, the solver implements an implicit finite difference scheme (Eulerian approach), and a Monte Carlo simulation (Lagrangian approach). The results are compared to one another, and also used to visualize the transport of the probability density function of the stock price. The latter of which can be directly related to the transported PDF of a the velocity of a fluid particle as obtained by the transported PDF method. The benefits and pitfalls of the different solution approaches are also explored. Finally, the paper concludes by discussing the real-world applications of this cross-disciplinary connection, and how advancements in either quantitative finance or computational fluid dynamics have potential to be utilized by the other.

To begin the comparison of the equations and methods, a foundational understanding of the Black-Scholes-Merton equation is necessary. [1]

II. Financial Derivatives and the Black-Scholes-Merton Equation

A. Financial Derivatives and Options

In order to understand the Black-Scholes-Merton equation, a foundational understanding of financial derivatives, specifically options, is required. According to Hull, a derivative is a “financial instrument whose value depends on (or derives from) the values of other, more basic, underlying variables” [1]. A stock option, for example, is a derivatve whose value is dependant on the price of a stock. However, different derivatives can be dependant on almost any variable, even the weather. For this paper, the primary derivatives of interest are options, which are securities that give “the right to buy or sell an asset, subject to certain conditions, within a specified period of time” [2].

The two major types of options are call and put options. A call option gives the holder the right, but not the obligation, to buy the underlying asset by a certain data for a certain price. A put option gives the holder the right, but not the obligation, to sell the underlying asset by a certain data for a certain price [1]. Options can also be further classified by when they can be

excercised. American options can be excercised at any time up to the expiration date as specified in the contract, while European options can only be excercised on the expiration date itself [2]. The price at which the asset may be bought or sold is refered to as the strike price, K, and the expiration date can an also be refrered to as the maturity date, T.

For the remainder of this paper, the focus will be on European call options. The value of a European call option at expiry can be determined by the following payoff function:

$$V = \max(S_T - K) \tag{1}$$

Where S_T is the stock price at expiry. From this payoff function one can observe that, if the stock price exceeds the strike price at expiry, the holder profits. However, if the stock price at expiry is below the strike price, the option expires worthless, as the holder is not obligated to execute the option. In this latter scenario, the holder only loses what they paid for the option. The Black-Scholes-Merton equation aims to determine a price for the contract, before its value is known at expiry.

B. The Black-Scholes-Merton Equation

In deriving their valuation formula, Black and Scholes [2] assume an idealized, frictionless market with continuous trading, no transaction costs, constant interest rates, no dividends, unrestricted short selling, and the ability to borrow or lend at the risk-free rate. The option is also assumed to be a European call option. Furthermore, an arbitrage-free market is also assumed, which means that no trading strategy can generate riskless profits. More explicitly, an arbitrage opportunity can be formally defined as "a (self-financing) trading strategy" that starts with zero value and terminates with a positive value. Therefore, a market is arbitrage-free if there are no such arbitrage opportunities [5].

The assumptions outlined above lead to the derivation of the following second-order linear parabolic Partial Differential Equation (PDE) for the price $V(S,t)$ of a European call option. The same equation was independently derived by Robert C. Merton [3]. Black, Scholes, and Merton all used stochastic calculus to obtain the equation. Nevertheless, the equation below is commonly referred to as the Black-Scholes-Merton PDE:

$$\frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} + rS\frac{\partial V}{\partial S} - rV = 0 \tag{2}$$

Where S is the underlying stock price, t is time, r is the risk-free interest rate, and σ is the volatility of the stock, a measure of how much the stock price fluctuates, or in other words a "measure of our uncertainty about the returns provided by the stock" [1]. This equation was obtained by forming a hedged portfolio that eliminates risk and requiring that its returns equal the risk-free rate in an arbitrage-free market.

It should also be noted that the structure of Equation 2 can be recognized as having the same form of the general transport equation given the context of this course. Where $\frac{\partial V}{\partial t}$ is the temporal term, $\frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2}$ is a diffusion term, $rS\frac{\partial V}{\partial S}$ is an advection term, and rV is a source (in this case, decay) term. This course has dedicated ample time to discritizing and computatonally solving equations of this exact form.

III. Derivation of the Black-Scholes-Merton Equation

The use of Stochastic Calculus in the derivation of Equation 2 is crucial to the argument of this paper. Therefore, this section will walk through such derivation as done so by Hull [1], Black and Scholes [2], and Merton [3].

A. Stochastic Calculus and Itô's Lemma

Before completing the derivation, it is important to highlight a few concepts from Stochastic Calculus that are used in the derivation. First, it should be noted that Stochastic Calculus is a branch of mathematics built specifically to handle the derivatives and integrals of stochastic processes, which are by definition, "an equation describing the probabilistic behavior of a stochastic variable" [1]. Furthermore, a stochastic variable can be defined as a variable "whose future is uncertain", or in other words, random. The first process to consider is the generalzied Wiener process:

$$dx = adt + bdz \tag{3}$$

Where $dz = \epsilon\sqrt{\Delta t}$ is refered to as a Wiener process, and ϵ has a standard normal distribution. Further, an Itô process is a generalized Wiener process where the parameters a and b are functions of the underlying variable and time, and can be expressed as:

$$dx = a(x,t)dt + b(x,t)dz \tag{4}$$

In standard calculus, the chain rule can be used to find the derivative or differential of a multivariable function. However, in Stochastic Calculus, one must use Itô's Lemma to compute the stochastic differential of a function that depends on a stochastic process. Doing so accounts for the fact that second-order terms can not be ignored in the formulation of the differential. This is because the diffusion term in a stochastic process (at least the ones within the scope of this paper) depends on a Wiener Process $dz \sim \sqrt{\Delta t}$, and when squared, this term is on the order of dt. Itô's lemma shows that for a function $G(x,t)$, where x is a stochastic variable following an Itô process, the differential of G can be expressed as

$$dG = \frac{\partial G}{\partial x}dx + \frac{\partial G}{\partial t}dt + \frac{1}{2}\frac{\partial^2 G}{\partial x^2}b^2 dt \tag{5}$$

Again, the key difference from ordinary calculus is the presence of the second-order term $\frac{\partial^2 G}{\partial x^2}b^2$ [1].

B. The Stock Price Model: Geometric Brownian Motion

Returning to the topics of this paper, the price of a stock S is modeled as a stochastic process using the following stochastic differential equation:

$$dS = \mu S\,dt + \sigma S\,dW \tag{6}$$

where μ is the stocks expected rate of return (under risk-neutral valuation, $\mu = r$, the risk-free rate), σ is the volatility of the stock price, and dW is a Wiener process (analagous to dz) [1], [6]. This model is commonly refered to as geometric Brownian motion. The first term, $rS\,dt$, represents the deterministic drift of the stock's price, and the second term, $\sigma S\,dW$ represents random variance.

C. Applying Itô's Lemma to the Option Value

Suppose $V(S,t)$ is the value of a European call option dependant on the price of stock S. Given that the price of S is modled using the Itô process above, Itô's lemma (Equation 5) can be applied, and the result can be rearranged using Equation 6 to obtain:

$$dV = \left(rS\frac{\partial V}{\partial S} + \frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2}\right)dt + \frac{\partial V}{\partial S}\sigma S\,dW \tag{7}$$

Since the Wiener processes in Equatiion 6 and Equation 7 are the same, a portfolion consisting of positions in both the option and the stock can be created such that the Wiener process is eliminated [1].

D. Constructing the Hedged Portfolio

This portfolio as created by Hull, consists of a short position (selling) in one derivative and a long posiiton (buying) in $\frac{\partial V}{\partial S}$ shares of the underlying stock.

$$\Pi = -V + \frac{\partial V}{\partial S} S \tag{8}$$

The change in this portfolio over a smal time interval, dt, can be expressed as:

$$d\Pi = -dV + \frac{\partial V}{\partial S}\, dS \tag{9}$$

The expressions above for dV (Equation 7) and dS (Equation 6) can be substituted into the equation above to obtain:

$$d\Pi = \left(-\frac{\partial V}{\partial t} - \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} \right) dt \tag{10}$$

As expected, the Wiener processes cancel, eliminating randomness fromt the expression.

E. No Arbitrage Condition

Hull states that because this expression is now free of the randomness induced by the Wiener process, it must be riskless. Therefore, due to the no arbitrage assumption, the portfolio must grow with the risk free rate:

$$d\Pi = r\, \Pi\, dt \tag{11}$$

Subsituting Equation 8 and Equation 10 then yields:

$$\left(\frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} \right) dt = r \left(V - \frac{\partial V}{\partial S} S \right) dt \tag{12}$$

Then, dividing both sides by dt and rearranging yields the Black-Scholes Merton PDE exactly as seen in Equation 2

$$\frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} + rS\frac{\partial V}{\partial S} - rV = 0 \tag{13}$$

IV. The Transported PDF Method

A. Statistical Description of Turbulent Flows

In *Turbulent Flows*, Stephen B. Pope remarks that "in a turbulent flow, the velocity field $U(x,t)$ is random" [4]. This means that if one were to repeat the same experiment multiple times under the same set of conditions, a given event may, but need not occur, making the event inherently unpredictable. Although a random variable, such as velocity, can not be predicted, the probability of events (e.g., the velocity is less than 10 m/s) can be used to create a Probability Density Function (PDF) that characterizes the random variable. Formally, the probability of any event can be determined by the Cumulative Distribution Function, and the PDF is defined to be the derivative of the CDF. Therefore, according to Pope, a random variable such as the velocity of a turbulent fluid, is completely characterized by its PDF, which can be thought of as the probability per unit distance in the sample space [4].

B. Eulerian PDF Transport Equation

The evolution of this Eulerian PDF of velocity in space and time can be represented by a transport equation derived by Pope from the Navier-Stokes equations. The result is the following Eulerian PDF transport equation:

$$\frac{\partial f}{\partial t} + V_i \frac{\partial f}{\partial x_i} = -\frac{\partial}{\partial V_i}\left[f \left\langle \frac{DU_i}{Dt} \middle| \mathbf{V} \right\rangle \right] \tag{14}$$

Where $f(\mathbf{V}; \mathbf{x}, t)$ is the Eulerian PDF of velocity.

C. The Generalized Langevin Model

To model the evolution of fluid particle velocities, Pope introduced the generalized Langevin model for a diffusion process. The Langevin equation was originally proposed in 1908 as a stochastic model for the velocity of a microscopic particle undergoing Brownian motion. Pope states that the equation provides a good model for the velocity of a fluid particle in turbulence [4]. The generalized Langevin model used to expresses the velocity of a fluid particle is expressed below:

$$d\mathbf{U}^*(t) = \mathbf{a}(\mathbf{U}^*(t), \mathbf{X}^*(t), t)\, dt + b(\mathbf{X}^*(t), t)\, d\mathbf{W} \tag{15}$$

where $\mathbf{a}(\mathbf{U}^*, \mathbf{X}^*, t)$ is the drift coefficient, $b(\mathbf{X}^*, t)$ is the diffusion coefficient, and $d\mathbf{W}$ is a vector-valued Wiener process [4]. The drift coefficient effectively describes the direction and speed of the system if no randomness were present, and the diffusion coefficient determines the intensity of the random fluctuations induced by the vector-valued Wiener process [4]. According to Pope, the Langevin equation is the simplest stochastic Lagrangian model for a turbulent fluid particle. It should also be noted that The stochastic process $U^*(t)$ is refered to in the text as the Ornstein-Uhlenbeck (OU) process, and although its PDF is already known to evolve by the Fokker-Planck equation [4], Pope still highlights this transition.

D. Derivation of the PDF Transport Equation

From the Langevin SDE (Equation 15), Itô calculus can be used to derive the Fokker-Planck equation, which is also known as the forward Kolmogorov equation, and describes the transport of the joint Lagrangian PDF of the particle velocity and position, $f_L^*(\mathbf{V}, \mathbf{x}; t)$. The Fokker-Planck equation can then be divided by the marginal position PDF, $f_X^*(\mathbf{x}, t)$ to obtain an expression for the evolution of the conditional PDF of the particle velocity, $f^*(\mathbf{V}|\mathbf{x}, t)$ [4].

$$\frac{\partial f^*}{\partial t} + V_i \frac{\partial f^*}{\partial x_i} = -\frac{\partial}{\partial V_i}\left[f^* a_i(\mathbf{V}, \mathbf{x}, t) \right] + \frac{1}{2} b(\mathbf{x}, t)^2 \frac{\partial^2 f^*}{\partial V_i \partial V_i} \tag{16}$$

Pope notes that the conditional PDF, f^* is the corresponding representation of the Eulerian PDF of velocity, f, for a particle system. Therefore, Equation 16 can be thought of as the Lagrangian equvalent of the Eulerian PDF transport equation, Equation 14. As a result, Equation 16 is commonly refered to as the Lagrangian PDF transport equation, and again, it describes the transport of the conditional PDF of the particle velocity [4].

V. The Connection Between the BSM Equation and the Transported PDF Method

A. Structural Equivalence of the SDEs

Both the Black-Scholes-Merton and Lagrangian PDF Transport equations are both derived from stochastic differential equations of the same drift plus diffusion form. Comparing the geometric Brownian motion SDE from which the BSM equation is derived (Equation 6):

$$dS = rS\,dt + \sigma S\,dW$$

with the generalized Langevin SDE (Equation 15) from which the Lagrangian PDF Transport equation is derived:

$$d\mathbf{U}^*(t) = \mathbf{a}(\mathbf{U}^*, \mathbf{X}^*, t)\,dt + b(\mathbf{X}^*, t)\,d\mathbf{W}$$

reveals that both equations have the same mathematical structure: deterministic drift term multiiplied by dt, plus a diffusion term (random fluctuations) driven by a Wiener process, dW. Furthermore, both equations can be derived using the same mathematical tools, Stochastic Calculus and Itô's lemma.

B. Variable Mapping

In short, the structural equivalece between the two mathematical frameworks can be summarized through the direct variable mapping:

TABLE I
Variable mapping between Black-Scholes-Merton and PDF Transport frameworks

Quantitative Finance (GBM / BSM)	**CFD (Langevin / PDF Transport)**
Stock price, S	Particle velocity, $\mathbf{U}^*$
Risk-free rate, r	Drift Coefficient, $\mathbf{a}$
Volatility, σ	Diffusion coefficient, b
Wiener process, dW	Vector-valued Wiener process, $d\mathbf{W}$
Monte Carlo simulation	Langrangian particle method

However, a few key distinctions should be made. The most prominent is that the BSM equation computes the conditional expectation of a future payoff given the current state, then discounts it back to give the current option price. Wheras, the Lagrangian Transported PDF equation describes how the PDF of velocity evolves over time. In other words, for BSM, the PDF of the stock price is used at expiry (or any future time) to find the expected payoff at that time, then the result is discount it back to its value at a past date. Another difference to highlight is that the drift and diffusion coefficients (r and σ) are constant for the Brownian Motion SDE, while the coefficients in the Langevin SDE ($\mathbf{a}$ and b) depend on the particle's velocity and position at a given time. This difference has a significant impact on the availale solution methods and validation techniques, which is discussed in Section VI. Finally, it should be mentioned that the stochastic differential equation for the velocity of a particle uses a vector-valued Wiener process, because the stochastic perturbations act in multiple spatial directions (typically three-dimensional space). Nevertheless, through observing the variable mapping above, a significant connection can be made between not just the two frameworks, but also their solution methods.

VI. Solution Methods

Both the Black-Scholes-Merton and Lagrangian Transported PDF equations can be solved using Eulerian (grid-based) and Lagrangian (particle-based) methods.

A. Eulerian Methods

The Eulerian approach solves the partial differential equation directly on a fixed grid. While Eulerian methods are efficient in low dimensions, and provide a deterministic solution, they scale poorly to high dimensions. For example, in order to simulate the price of a portfolio of two stocks, if each stocks price axis was discritized using 500 points, the solver would require 250,000 grid locations to cover all possible combinations. The same cocnept can be applied to the transported PDF method for high-dimensional PDFs. This is the exact reason, when refering to the velocity-frequency joint PDF, that Pope states is not feasible, computationally, to represent the joint PDF accurately through a discretization of the seven-dimensional V-θ-x space, as is required in finite-difference, finite-volume, and finite-element methods [4]. Therefore, the transported PDF equations are modeled using langragian methods.

B. Lagrangian Methods

The Lagrangian approach simulates random particle trajectories using the respective underlying stochastic differential equation, then uses statistics to obtain the desired information from the resulting data. The Lagrangian approach most typically used to solve the Black-Scholes-Merton equation is the Monte Carlo method. For thsi, many random stock price paths are simulated using the Geometric Brownian Motion SDE. The payoff is computed for each path, and the option value is estimated as the average payoff discounted back to the current time. For the transported PDF method, the Lagrangian particle method is used. In this case, the velocities of many fluid particles in a turbulent flow are simulated using the Langevin SDE, and the resuting PDF and its transport is constructed from the particle statistics [4].

The primary advantage of Lagrangian methods is that they scale well with high dimensions. Adding additional state variables (e.g., more stocks or velocity compnents) just means more longer vectors are simulated. In this case, the computational cost grows linearly with added dimensions, whereas with Eulerian methods it grows exponentially. Furthermore, since each path is independent from one another, Lagrangian methods are more compatible with GPUs and High Performance Clusters, which can parallelize computations across many processing units. The primary limitation of Lagrangian models is statistical noise. The standard error of a Motne Carlo analysis is proportional to $1/\sqrt{N}$, where N is the number of simulated paths [1]. Therefore, to cut the error in half, four times as many paths are needed, and to reduce the error by a factor of ten, 100 times as many paths are required

C. Constant vs. Variable Coefficients

As mentioned previously, the coefficients of the Geometric Brownian Motion and Langevin SDEs have a significant impact on the available solution methods. The constant coefficients of the Geometric Brownian Motion SDE allow for Itô's lemma to be aplied to $\ln(S)$, which shows that $\ln(S)$ follows a generalized Wiener process, and further that $\ln(S_T)$ is normally distributed. This is significant for two reasons. First, because $\ln(S_t)$ is normally distributed, then S_t must

be lognormally distributed. Therefore, the standard equation for the probability density function of a lognormal variable can be used [7].

$$f(S_t|t) = \frac{1}{S_t\omega\sqrt{2\pi}} \exp\left[-\frac{(\ln(S_t) - \theta)^2}{2\omega^2}\right] \tag{17}$$

Where $\theta(t) = \ln(S_0) + (r - \sigma^2/2)t$ and $\omega(t) = \sigma\sqrt{t}$ are the mean and standard deviation of $\ln(S)$ respectively, and t is the current time step. Second, it allows for an exact step formula for the SDE to be obtained [1].

$$S(t + \Delta t) = S(t) \cdot \exp\left[\left(r - \frac{1}{2}\sigma^2\right)\Delta t + \sigma\epsilon\sqrt{\Delta t}\right] \tag{18}$$

Where ϵ is a random sample from a normal distribution with a mean of 0 and a standard deviation of 1.0. For the Langevin SDE, because the coefficients $\mathbf{a}$ and b depend on the particle's velocity and position at a given time, the Langevin SDE must be discritized directly:

$$\mathbf{U}^*(t + \Delta t) - \mathbf{U}^*(t) = \mathbf{a}(\mathbf{U}^*, \mathbf{X}^*, t)\,\Delta t + b(\mathbf{X}^*, t)\,\Delta\mathbf{W} \tag{19}$$

Furthermore, the standard equation for the probability density function of a lognormal variable can no longer be utilized, and the PDF must be constructed using simulated particle data.

VII. Computational Experiments and Results

To further demonstrate the connection between the BSM equation and the transported PDF method, and to explore the Eulerian and Lagrangian solution methods, a Python script was developed that utilizes three methods for the BSM equation.

A. Problem Description

The script using the followg parameters for all implented solution methods:

TABLE II
PARAMETERS USED FOR EUROPEAN CALL OPTION PRICING.

Parameter	**Symbol**	**Value**
Strike price	K	\$100
Risk-free rate	r	5%
Volatility	σ	20%
Time to expiry	T	1 year

B. Analytical Solution

The analytical solution to the Black-Scholes formula for a European call option is [2]:

$$V = S\,N((d_1) - Ke^{-rT}\,N(d_2)) \tag{20}$$

where N is the standard normal cumulative distrobution function and:

$$d_1 = \frac{\ln(S/K) + (r + \frac{1}{2}\sigma^2)(T - t)}{\sigma\sqrt{T - t}}, \qquad d_2 = \frac{\ln(S/K) + (r - \frac{1}{2}\sigma^2)(T - t)}{\sigma\sqrt{T - t}} \tag{21}$$

where t is the current time. This formula provides the exact solution and can be used to validate the other numerical methods outlined in this paper.

C. Finite Difference Solver (Eulerian)

The BSM PDE (Equation 2) was discritized using an implicit finite diference scheme. The stock price axis was discretized into a uniform grid with 500 intervals from $S = \$0$ to $S_{\max} = \$500$, and time was discritized into 5000 steps marching backwards from $t = T$ to $t = 0$. Second order central difference stencils were used for the first and second spatial derivatves, and a forward difference was used for the temporal term.

After substituting the stencils above into the PDE, and by writing S_j as ΔS, a trigiagonal system of equations is obtained for each time step. This tridiagonal system has the general form of:

$$a_j V_{j-1}^i + b_j V_j^i + c_j V_{j+1}^i = V_j^{i+1} \tag{22}$$

where the coefficients a_j, b_j, and c_j are expressed as:

$$a_j = \frac{\Delta t}{2}\left(rj - \sigma^2 j^2\right), \quad b_j = 1 + \Delta t\left(\sigma^2 j^2 + r\right), \quad c_j = -\frac{\Delta t}{2}\left(\sigma^2 j^2 + rj\right) \tag{23}$$

It should be noted, that since this scheme marches backawards in time, the spatial terms are evaluated at the unknown time step (i), therefore this scheme is implicit and unconditionally stable. The terminal condition is the payoff function $V_j^N = \max(j\Delta S - K, 0)$, the boundary condition at the bottom bound of the stock price axis is ($V(0,t) = 0$), and the boundary condition at the top bound of the stock price axis is $V(S_{max}, t) = S_{\max} - Ke^{-r(T-t)}$. For this example, with $S_{max} = \$500$, the strike price $K = \$100$ is roughly 8 standard deviations below the mean of the stock price distribution in log space. Since the probability of this event (the stock price dropping below the strike price) is essentially zero, it is assumed that the option will be exercised at expiry, and the boundary condition $V(S_{max}, t) = S_{\max} - Ke^{-r(T-t)}$ is valid. The tridiagonal system was solved at eachtime step using a banded solver, produycing the option value at each grid point for all time steps.

D. Monte Carlo Solver (Lagrangian)

The Monte Carlo simulation was implemented using the exact step formula for the Geometric Brownian motion SDE (Equation 18). 200,000 random paths were simulated to the final time T for 80 stock prices between \$50 and \$200. The value of the option at $t = 0$ can then be obtained using the risk-neautral valuation formula, by discounting the expected (average) payoff at maturity back to the current time step at the risk-free rate [1].

$$V = e^{-rT}\,\hat{E}\left[\max(S_T - K, 0)\right] \tag{24}$$

E. Results: Option Pricing

Figure 1 shows the option value V as a function of the current stock price S for all three solution methods. The analytical Black-Scholes formula (solid line), the finite difference solver (req squares), and the Monte Carlo simulation (blue circles with error bars) all produce identical results, reinforcing the argument that Eulerian and Lagrangian solution methods produce consistant, accurate results.

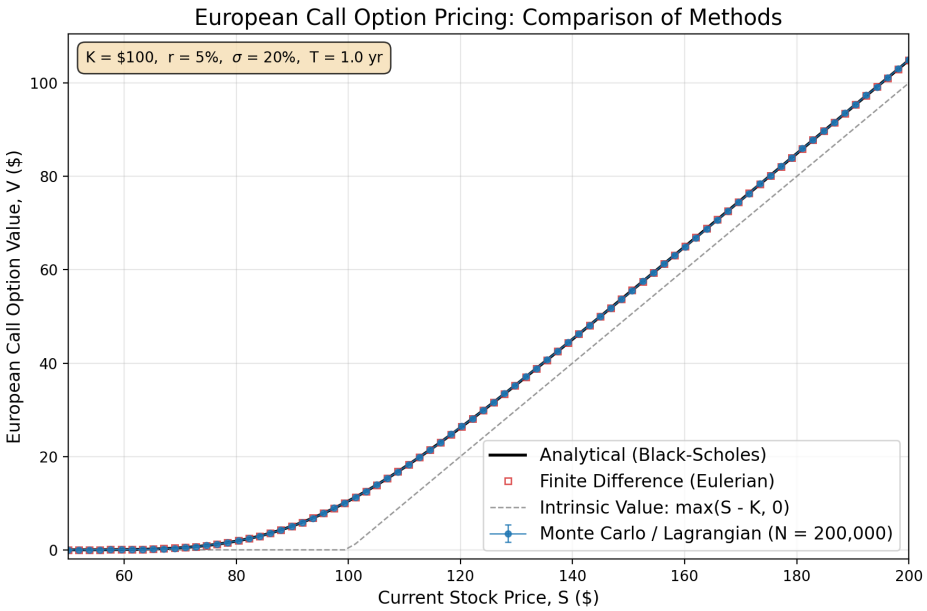

Fig. 1. European call option value vs. stock price: comparison of analytical, eulerian, and lagrangian methods

F. Results: Monte Carlo Visualization

To visualize the stochastic nature of the Monte Carlo simulation, Figure 2 shows 30 individual stock price paths sampled from 100 simulated paths, all starting at $S_0 = \$100$. The difussion of the paths over time demonstrates the effect of volatility. The bold blue line represents the mean of the 100 simulated paths, and the black dashed line represents the theoretircal expected stock price, $E(S_t) = S_0 e^{rt}$ [1]. As the number of simulated paths increases (and with this the accuracy of the Monte Carlo simulation), the mean converges to this theoretical value.

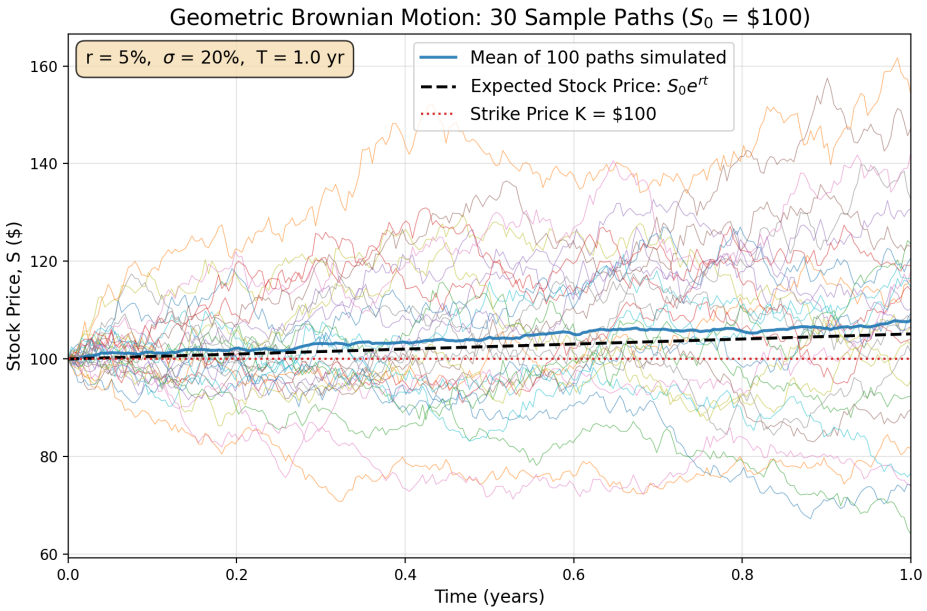

Fig. 2. Geometric Brownian Motion: 30 sample paths with mean and expected return.

G. Results: PDF Transport

The data from the Monte Carlo simulation was then used to construct the probability density function of the stock price at each time step. This is the step that reinforces the mapping between Monte Carlo simulations and Lagrangian particle methods. For a turbulent flow, particle paths would be simualted using the Langevin SDE, their velocities (conditioned on position and time) would be recorded at each time step, and the PDF surface would be constructed from the statistics.

Figure 3 shows the 3D PDF evolution surface for s tock with initial price $S_0 = \$100$. The surface was constructed entirely from Monte Caro particle data, as 200,000 simulated paths were used to create a histogram at selected time steps. The PDF starts as a sharp spike near the initial stock price, then shifts and spreads over time due to drift and diffusion.

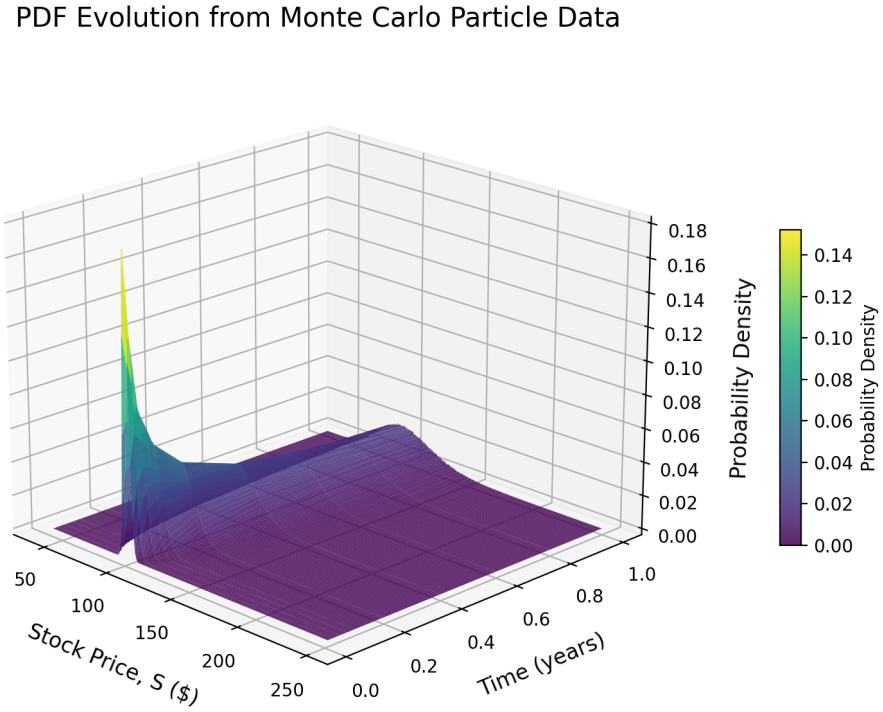

Fig. 3. PDF evolution from Monte Carlo particle data

Figure 4 provides a clearer view of the drift and diffusion of the PDF over time. It also further demonstrates the accuracy of the Monte Carlo simulation by comparing its results to the analytical expression for the probability density function of a lognormal variable (Equation 17'). The shaded bars are the PDFs constructed from the Monte Carlo data plotted as histograms, and the solid curves are the analytical lognormal PDFs. Over time, the volatility spreads the distribution of possible stock prices (diffusion), and the drift shifts it to the right (advection). For a turbulent flow, drift can be thought of as where the average velocity trends, and diffusion as the effect of turbulence randomizing the velocity around the mean.

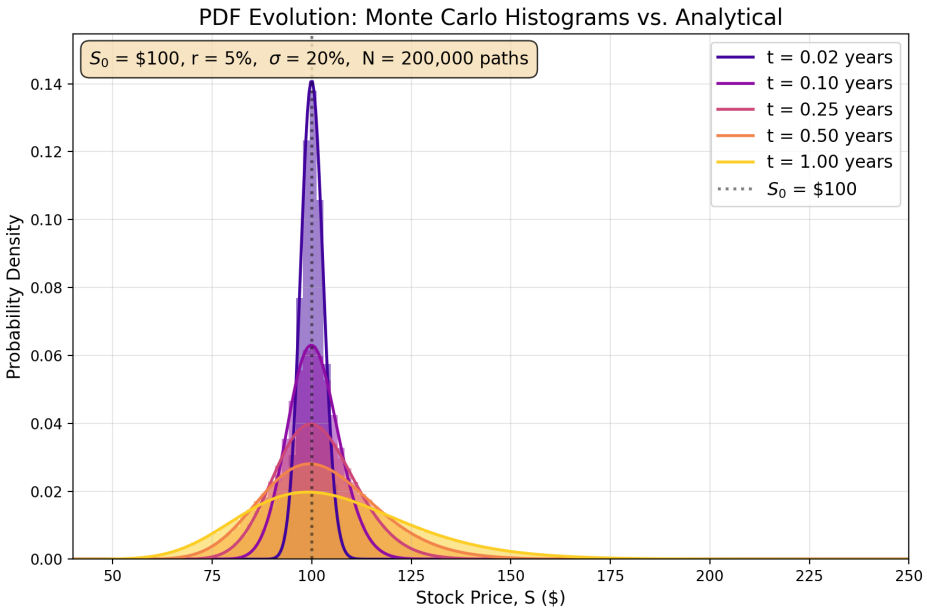

Fig. 4. PDF Evolution: Monte Carlo histograms vs. log-normal PDF.

Figure 5 shows a top-down heatmap view of the PDF evolution, with the expected stock price $E(S_t) = S_0 e^{rt}$ (white dashed line) and the $\pm 1\sigma$ bounds (white dotted lines) overlaid. The drift of the PDF is clearly visible, aligning with the expected stock price, which can also be thought as the deterministic dorwth of the stock in the absence of volatility.

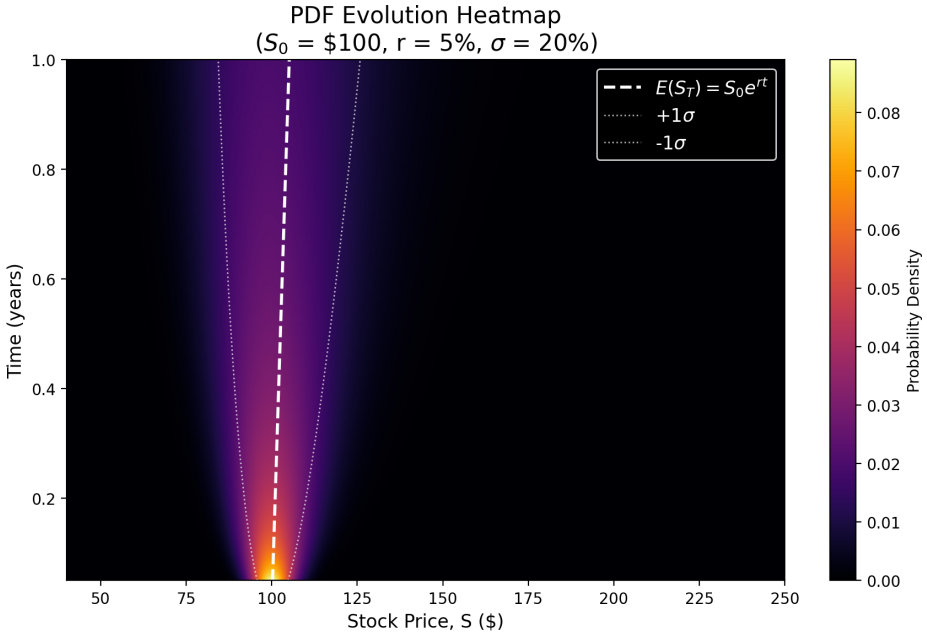

Fig. 5. PDF evolution heatmap with drift path and $\pm 1\sigma$ bounds.

H. Results: Monte Carlo Convergence

Finally, Figure 6 demonstrates the convergance behavior of the Monte Carlo method. The standard error of the estimate produced by the simulation is expressed as $\omega/\sqrt{(N)}$ [1]. Where ω is the standard deviation of the discounted payoffs, and N is the number of simulated paths. With 100 paths, the estimate is noisy because each path has a greater weight. However, by 1,000,000 paths, the estimate approaches the exact value of obtained using the Black-Scholes formula. As mentioned in Section VI, this slow convergance rate is a major limitation of Lagrancien approches.

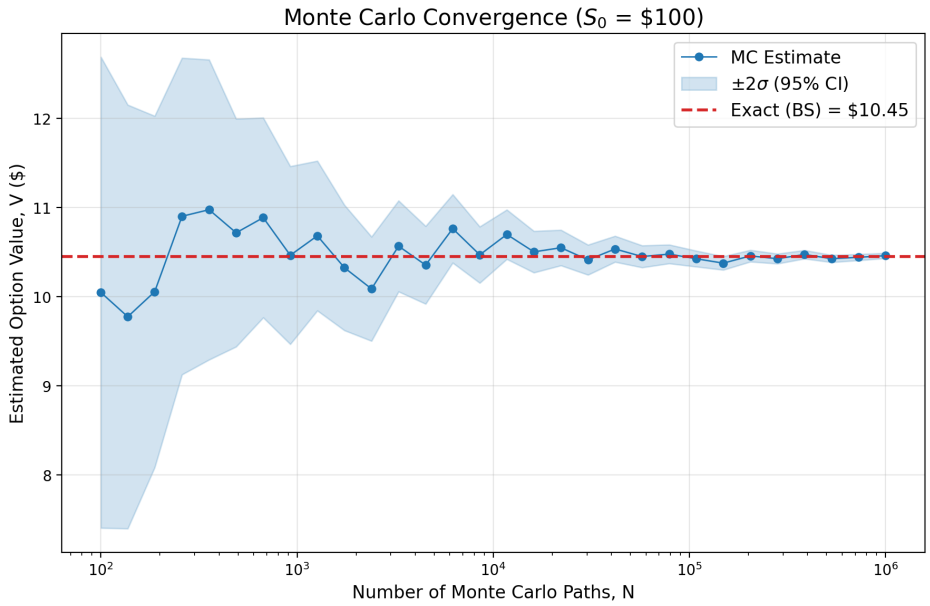

Fig. 6. Monte Carlo convergence: estimated option value vs. number of simulated paths.

VIII. Conclusions

The contents of this paper and the results of the computational experimants have demonstrated the mathematical connection between the Black-Scholes-Merton equation for pricing financial derivatives, and the transported probability density function method used in computational fluid dynamics. From the material presented in this paper, several conclusions can be drawn.

First, the BSM PDE and the Fokker-Plank equation (forward Kolmogorov) are both derived from SDEs of the same drift-diffusion form, establishing a direct mathematical connection between financial derivative pricing and the transported PDF method in CFD.

Second, both Eulerian (finite difference) and Lagrangian (Monte Carlo) methods produce results consistant with not ony eachother, but also analytical solutions. This was validated by the agreement between the numerical results obtained through the experimental Eulerian and Lagrangian solvers andthe exact values obtained using the analytical Black-Scholes pricing formula.

Third, Monte Carlo particle simulations can reproduce results consistent with analytical probability desnity functions, as demonstrated in Section VII-G and Figure 4. This is an extremely important conclusion in the case of the transported PDF method, where no analytical probability density function exits.

Fourth, in the case of the Black-Scholes-Merton equation, an analytical solution exists for the Geometric Brownian Motion SDE of a stock price which implies that a stocks price at a future time (given its initial price) is lognormally distributed. As a result, the analytical expression for the probabillity densitiy function of a lognormal random variable can be used to obtain the PDF. Therefore, the BSM equation can be used to validate Lagrangian particle methods, because the PDFs generated from the particle methods can be compared with those obtained analytically, as demonstrated with the reslts of the Monte Carlo experiment discussed in this paper.

Finally, advancements in fincncial modeling and computational fluid dynamics are mutually beneficial. The shared mathematical framework illustrated in this paper means that numerical methods, variance reduction techniques, and other solution strategies developed in thefields of quantatative finance and computational fluid dynamics can possibly be utlilized or even directly applied to problems in the other.

REFERENCES

[1] J. C. Hull, *Options, Futures, and Other Derivatives*, 10th ed. Boston, MA: Pearson, 2022.

[2] F. Black and M. Scholes, "The pricing of options and corporate liabilities," *Journal of Political Economy*, vol. 81, no. 3, pp. 637–654, 1973.

[3] R. C. Merton, "Theory of rational option pricing," *Bell Journal of Economics and Management Science*, vol. 4, no. 1, pp. 141–183, 1973.

[4] S. B. Pope, *Turbulent Flows*. Cambridge: Cambridge University Press, 2000.

[5] M. Baxter and A. Rennie, *Financial Calculus: An Introduction to Derivative Pricing*. Cambridge: Cambridge University Press, 1996.

[6] S. E. Shreve, *Stochastic Calculus for Finance II: Continuous-Time Models*. New York, NY: Springer, 2004.

[7] D. C. Montgomery, G. C. Runger, and N. F. Hubele, *Engineering Statistics*, 5th ed. Hoboken, NJ: John Wiley & Sons, 2011.

APPENDIX

```python
"""
Christian DiPietrantonio
ME 5311: Computational Methods to Viscous Flows
Term Paper
04/19/2026
-------------------------------------------------------------------------
Black-Scholes-Merton Solver
-------------------------------------------------------------------------

This script demonstrates three approaches to pricing a European call option:
   1. Analytical Black-Scholes formula (closed-form solution)
   2. Monte Carlo simulation (Lagrangian / stochastic particle method)
   3. Finite Difference (Eulerian / grid-based PDE solver)

Methods 2 and 3 are the Lagrangian and Eulerian numerical approaches.
"""

import numpy as np
from scipy.stats import norm
from scipy.linalg import solve_banded
import matplotlib.pyplot as plt
from matplotlib import cm
from mpl_toolkits.mplot3d import Axes3D
import os

# -------------------------------------------------------------------------
# Parameters
# -------------------------------------------------------------------------
K       = 100.0 # Strike Price ($)
r       = 0.05 # Risk-free interest rate (5%)
sigma   = 0.2 # Volatility
T       = 1.0 # Time to expiry (1 year)

# Monte Carlo Settings
N_paths = 200000   # Number of simulated paths
N_steps = 252      # Time steps (trading days in a year)
N_S_pts = 80       # Number of starting prices to evaluate


# Stock price range
S_min = 50
S_max = 200

# Output directory
out_dir = "figures"
os.makedirs(out_dir, exist_ok=True)

# -------------------------------------------------------------------------
# Analytical Black-Scholes Formula
# -------------------------------------------------------------------------
def bs_call(S, K, r, sigma, T):
   """
   Analytical Black-Scholes price for a European call option.
   Page 644, Black-Scholes
   Page 91, Baxter-Rennie
   Page 336, Hull

   Parameters
   ----------
```

```
    S    : current stock price
    K    : strike price
    r    : risk free rate
    sigma : volatility
    T    : time to expiration

    Returns
    -------
    V    : option prices
    """

    d1 = (np.log(S / K) + (r + 0.5 * sigma**2) * T) / (sigma * np.sqrt(T))
    d2 = d1 - sigma * np.sqrt(T)
    V = S * norm.cdf(d1) - K * np.exp(-r * T) * norm.cdf(d2)

    return V

# ------------------------------------------------------------------------
# Monte Carlo Solver - Terminal Value Method
# ------------------------------------------------------------------------
def mc_option_price(S0, K, r, sigma, T, N_paths):
    """
    Price a European call via Monte Carlo using the exact GBM solution. Instead of
        stepping through time, we jump diretly to S(T). This works because we only need
        the terminal stock price to compute the payoff. For the PDF evolution, we must
        use the full time-steppind version below.

    Page 217, Hull (Long position European Call option payoff)
    Page 336, Hull (Option expected value discounted at risk-free rate)
    Page 471, Hull (Stock price at expiration time)
    Page 473, Hull (Standard Error of estimate)


    Parameters
    ----------
    S0    : starting stock price
    K     : strike price
    r     : risk free rate
    sigma : volatility
    T     : time to expiration
    N_paths : number of simulated paths

    Returns
    -------
    V_mean : estimated option price/value
    V_std : standard error of the estimate

    """

    Z = np.random.standard_normal(N_paths)
    ST = S0 * np.exp((r - 0.5 * sigma**2) * T + sigma * np.sqrt(T) * Z)

    # payoff for a European call
    payoffs = np.maximum(ST - K, 0.0)

    # discount back to present value
    V_mean = np.exp(-r * T) * np.mean(payoffs)
    V_std = np.exp(-r * T) * np.std(payoffs) / np.sqrt(N_paths)

    return V_mean, V_std

```

```python
# ---------------------------------------------------------------------
# Monte Carlo Solver - Full Path Simulation
# ---------------------------------------------------------------------
def mc_full_paths(S0, r, sigma, T, N_paths, N_steps):
    """
    Simulate full GBM paths.

    Page 470, Hull (Discritizing SDEs)
    Note:
        - For this example we use the more accurate analytical solution for the GBM SDE.
            For the transported PDF method, we would discritize the SDE itself.
    Page 471, Hull (Stock price at time step)

    Parameters
    ----------
    S0    : starting stock price
    r     : risk free rate
    sigma : volatility
    T     : time to expiration
    N_paths : number of simulated paths
    N_steps : number of time steps

    Returns
    -------
    S     : all price paths
    t     : time array

    """

    dt = T / N_steps
    t = np.linspace(0, T, N_steps + 1)

    S = np.zeros((N_steps + 1, N_paths))
    S[0, :] = S0

    # generate random increments
    Z = np.random.standard_normal((N_steps, N_paths))

    # step through time
    for i in range(N_steps):
        S[i+1, :] = S[i, :] * np.exp((r - 0.5 * sigma**2) * dt + sigma * np.sqrt(dt) * Z
            [i, :])

    return S, t

# ---------------------------------------------------------------------
# Finite Difference Solver (Eulerian / Grid Based PDE Solver)
# ---------------------------------------------------------------------
def fd_bsm(K, r, sigma, T, S_max=500.0, M=500, N_t=5000):
    """
    Solver the BSM PDE using implicit finite difference method.

    This is the Eulerian approach: discritize stock price into a grid and solve the PDE
         directly at each point. Important to note that we march backwards in time from
        t = T (expiry) to t=0 (today).

    Parameters
    ----------
    K     : strike price
    r     : risk free rate
    sigma : volatility
```

```
    T     : time to expiration
    S_max : upper bound of stock price grid
    M     : number of spacial intervals
    N_t   : number of time steps

    Returns
    -------
    S_grid : stock price at grid points
    V      : option values at t = 0 (today)
    """

    # ----- Grid Setup -----
    dS    = S_max / M     # spatial step size
    dt    = T / N_t       # time step size
    S_grid = np.linspace(0, S_max, M + 1)

    # ----- Terminal Condition -----
    # At expiry, option value = payoff = max(S - K, 0)
    V = np.maximum(S_grid - K, 0.0)

    # ----- Build Tridiagonal Coefficients for Interior Nodes j = 1, ..., M - 1 -----
    j = np.arange(1, M)

    a_j = 0.5 * dt * (r * j - sigma**2 * j**2)
    b_j = 1.0 + dt * (sigma**2 * j**2 + r)
    c_j = -0.5 * dt * (sigma**2 * j**2 + r * j)

    # ----- Format into Banded System -----
    n_interior = M - 1 # number of unknowns
    # Banded matrix (l + u + 1, n) where l = 1 lower diagonal, u = 1 upper diagonal
    # element[row, col] maps to ab[u + row - col, col]
    ab = np.zeros((3, n_interior))
    ab[0, 1:] = c_j[:-1]
    ab[1,:] = b_j
    ab[2, :-1] = a_j[1:]

    # ----- Time Marching -----
    for n in range(N_t):
        rhs = V[1:M].copy()

        time_to_expiry = (n + 1) * dt
        V_top = S_max - K * np.exp(-r * time_to_expiry)

        rhs[-1] -= c_j[-1] * V_top

        # Solve the Trigiagonal System
        V[1:M] = solve_banded((1, 1), ab, rhs)

        # Apply boundary condtions to full solution array
        V[0] = 0.0
        V[M] = V_top

    return S_grid, V

# ------------------------------------------------------------------------
# Figure 1: V(S) Comparison
# ------------------------------------------------------------------------
def plot_option_value_curve():
    """
    Compare Monte Carlo, Finite Difference, and Analytical Option Prices.
    """
```

```
236
237     print("Computing V(S) curve ...")
238     S_arr = np.linspace(S_min, S_max, N_S_pts)
239
240     #Analytical Solution
241     V_analytical = bs_call(S_arr, K, r, sigma, T)
242
243     # Monte Carlo (loop over starting prices)
244     V_mc      = np.zeros(N_S_pts)
245     V_err     = np.zeros(N_S_pts)
246
247     for i, S0 in enumerate(S_arr):
248        V_mc[i], V_err[i] = mc_option_price(S0, K, r, sigma, T, N_paths)
249
250     # Finite Difference
251     print("  Running finite difference solver...")
252     S_fd, V_fd = fd_bsm(K, r, sigma, T, S_max=500.0, M=500, N_t=5000)
253
254     # Interpolate FD results onto same S_arr for comparison
255     V_fd_interp = np.interp(S_arr, S_fd, V_fd)
256
257     # ----- Plot -----
258     fig, ax = plt.subplots(figsize=(9, 6))
259
260     # Analytical Solution
261     ax.plot(S_arr, V_analytical, 'k-', linewidth = 2, label='Analytical (Black-Scholes)
            ')
262
263     # Finite Difference Solution
264     ax.plot(S_arr, V_fd_interp, 's', color='tab:red', markersize=4, markerfacecolor='
            none', markeredgewidth=1.0, label='Finite Difference (Eulerian)', alpha=0.8)
265
266     # Monte Carlo (with error bars)
267     ax.errorbar(S_arr, V_mc, yerr=4*V_err, marker='o', color='tab:blue', markersize=4,
            linewidth =1.0, capsize=2, label=f'Monte Carlo / Lagrangian (N = {N_paths:,})',
            alpha=0.8)
268
269     # Intrinsic Value for Reference
270     intrinsic = np.maximum(S_arr - K, 0)
271     ax.plot(S_arr, intrinsic, '--', color='tab:gray', linewidth=1.0, alpha=0.8, label='
            Intrinsic Value: max(S - K, 0)')
272
273     ax.set_xlabel('Current Stock Price, S ($)', fontsize=12)
274     ax.set_ylabel('European Call Option Value, V ($)', fontsize=12)
275     ax.set_title('European Call Option Pricing: Comparison of Methods', fontsize=15)
276     ax.legend(fontsize=12)
277     ax.set_xlim(S_min, S_max)
278     ax.set_ylim(bottom=-2)
279     ax.grid(True, alpha=0.3)
280
281     param_text = (fr'K = \${K:.0f}, r = {r:.0%}, $\sigma$ = {sigma:.0%}, T = {T:.1f} yr
            ')
282     ax.text(0.02, 0.97, param_text, transform=ax.transAxes, fontsize=10,
            verticalalignment='top', bbox=dict(boxstyle='round', facecolor='wheat',
            edgecolor='black', pad=0.5, alpha=0.8))
283
284     fig.tight_layout()
285     fig.savefig(f'{out_dir}/figure01_option_value_curve.png', dpi=200, bbox_inches="
            tight")
286     plt.close(fig)
287     print(f'  Saved: {out_dir}/figure01_option_value_curve.png')
```

```
288
289     return S_arr, V_analytical, V_mc, V_err, V_fd_interp
290
291 # ----------------------------------------------------------------------
292 # Figure 2: Sample GBM Paths
293 # ----------------------------------------------------------------------
294 def plot_sample_paths(S0=100.0, N_show=30, N_sim=100):
295     """
296     Show a handful of GBM trajectories to illustrate stochastic behavior.
297
298     Page 323, 325, Hull (Expected Return)
299     """
300
301     print("Simulating sample paths...")
302     S, t = mc_full_paths(S0, r, sigma, T, N_sim, N_steps)
303
304     fig, ax = plt.subplots(figsize=(9, 6))
305
306     # plot subset of paths
307     for j in range(N_show):
308         ax.plot(t, S[:, j], linewidth=0.5, alpha=0.6)
309
310     # overlay the mean of the simulated paths
311     S_mean_sim = np.mean(S, axis=1)
312     ax.plot(t, S_mean_sim, color='tab:blue', linewidth=2.0, alpha=0.9, label=f'Mean of
            {N_sim} paths simulated')
313
314     # overlay the expected return
315     S_expected = S0 * np.exp(r * t)
316     ax.plot(t, S_expected, 'k--', linewidth=2.0, label=f'Expected Stock Price: $S_0 e
            ^{{rt}}$')
317
318     ax.axhline(K, color='tab:red', linestyle=':', linewidth=1.5, label=f'Strike Price K
             = ${K:.0f}')
319
320     ax.set_xlabel('Time (years)', fontsize=12)
321     ax.set_ylabel('Stock Price, S ($)', fontsize=12)
322     ax.set_title(fr'Geometric Brownian Motion: {N_show} Sample Paths ($S_0$ = \${S0:.0f
            })', fontsize=15)
323     ax.legend(fontsize=12, loc='upper center', bbox_to_anchor=(0.6, 1.0))
324     ax.set_xlim(0, T)
325     ax.grid(True, alpha=0.3)
326
327     param_text = (fr'r = {r:.0%}, $\sigma$ = {sigma:.0%}, T = {T:.1f} yr')
328     ax.text(0.02, 0.97, param_text, transform=ax.transAxes, fontsize=12,
            verticalalignment='top', bbox=dict(boxstyle='round', facecolor='wheat',
            edgecolor='black', pad=0.5, alpha=0.8))
329
330     fig.tight_layout()
331     fig.savefig(f'{out_dir}/figure02_GMB_sample_paths.png', dpi=200, bbox_inches="tight
            ")
332     plt.close(fig)
333     print(f'  Saved: {out_dir}/figure02_GMB_sample_paths.png')
334
335     return S, t
336
337 # ----------------------------------------------------------------------
338 # Figure 3: PDF Evolution - 3D Surface
339 # ----------------------------------------------------------------------
340
341 def plot_pdf_evolution(S0=100.0, N_sim=200000):
```

```
    """
    Visualize how the probability density of a stock price evolves over time using
        Monte Carlo particle data.
    Note:
        - The exact GBM sol could be used to construct the PDF using the PDF function
            for a lognormal distribution. However, here the PDF is constructed using MC
            data, because no PDF function is avalable for the Langevin SDE.

    Page 322, Hull (Mean and Standard Deviation of ln ST)
    Page 84, Montgomery, Runger, Hubele
    """

    print("Computing PDF evolution surface from Monte Carlo data...")

    t_snapshots = np.array([0.01, 0.025, 0.05, 0.075, 0.1, 0.2, 0.4, 0.6, 0.8, 1.0])
    S_bins = np.linspace(40, 250, 500)
    S_bin_centers = 0.5 * (S_bins[:-1] + S_bins[1:])

    # Simulate paths
    S_all, t_all = mc_full_paths(S0, r, sigma, T, N_sim, N_steps)

    # Build PDF at each time snapshot
    PDF_mc = np.zeros((len(t_snapshots), len(S_bin_centers)))
    for i, t_val in enumerate(t_snapshots):
       idx = np.argmin(np.abs(t_all - t_val))
       counts, _ = np.histogram(S_all[idx, :], bins=S_bins, density=True)
       PDF_mc[i, :] = counts

    fig = plt.figure(figsize=(9, 6))
    ax = fig.add_subplot(111, projection='3d')
    T_mesh, S_mesh = np.meshgrid(t_snapshots, S_bin_centers, indexing='ij')
    surf = ax.plot_surface(S_mesh, T_mesh, PDF_mc, cmap='viridis', alpha=0.85, rstride
        =1, cstride=3, edgecolor='none')

    ax.set_xlabel('Stock Price, S ($)', fontsize=12, labelpad=10)
    ax.set_ylabel('Time (years)', fontsize=12, labelpad=10)
    ax.set_zlabel('Probability Density', fontsize=12, labelpad=10)
    ax.set_title('PDF Evolution from Monte Carlo Particle Data', fontsize=15)
    ax.view_init(elev=20, azim=-45)
    fig.colorbar(surf, ax=ax, shrink=0.45, aspect=15, label ='Probability Density', pad
        =0.10)

    fig.tight_layout()
    fig.savefig(f'{out_dir}/figure03_3d_pdf_evolution.png', dpi=200, bbox_inches="tight
        ")
    plt.close(fig)
    print(f'  Saved: {out_dir}/figure03_3d_pdf_evolution.png')
    return t_snapshots, S_bin_centers, PDF_mc

# ------------------------------------------------------------------------
# Figure 4: PDF Evolution - 2D Slices
# ------------------------------------------------------------------------
def plot_pdf_slices(S0=100.0, N_sim=200000):
    """
    Monte Carlo histograms overlaid with analytical log-normal curves. This deomstrates
         that the Lagrangian statistics converge to the Eulerian PDF.

    Page 322, Hull (Mean and Standard Deviation of ln ST)
    Page 84, Montgomery, Runger, Hubele
    """

```

```
    print("Computing PDF time slices...")

    t_slices = [0.02, 0.1, 0.25, 0.5, 1.0]
    S_grid = np.linspace(40, 250, 500)

    # Simulate paths
    S_all, t_all = mc_full_paths(S0, r, sigma, T, N_sim, N_steps)

    colors = cm.plasma(np.linspace(0.1, 0.9, len(t_slices)))

    fig, ax = plt.subplots(figsize=(9, 6))
    pdf_max = 0.0

    for i, t_val in enumerate(t_slices):
        # Monte Carlo Histogram
        idx = np.argmin(np.abs(t_all - t_val))
        ax.hist(S_all[idx, :], bins=120, range=(40, 250), density=True, alpha=0.5, color
            =colors[i], edgecolor='none')

        # Analytical log-normal PDF
        mu_ln = np.log(S0) + (r - 0.5 * sigma**2) * t_val
        sigma_ln = sigma * np.sqrt(t_val)
        pdf_vals = (1.0 / (S_grid * sigma_ln * np.sqrt(2*np.pi)) * np.exp(-0.5 * ((np.
            log(S_grid) - mu_ln) / sigma_ln)**2))
        pdf_max = max(pdf_max, np.max(pdf_vals))
        ax.plot(S_grid, pdf_vals, color=colors[i], linewidth=2.0, label=f't = {t_val:.2f
            } years')

    ax.axvline(S0, color='black', linestyle=':', linewidth=2.0, alpha=0.5, label=fr'
        $S_0$ = \${S0:.0f}')

    ax.set_xlabel('Stock Price, S ($)', fontsize=12)
    ax.set_ylabel('Probability Density', fontsize=12)
    ax.set_title('PDF Evolution: Monte Carlo Histograms vs. Analytical', fontsize=15)
    ax.legend(fontsize=12, loc='upper right')
    ax.set_xlim(40, 250)
    ax.set_ylim(0, pdf_max * 1.1)
    ax.grid(True, alpha=0.3)

    param_text = (fr'$S_0$ = \${S0:.0f}, r = {r:.0%}, $\sigma$ = {sigma:.0%}, N = {
        N_sim:,} paths')
    ax.text(0.02, 0.97, param_text, transform=ax.transAxes, fontsize=12,
        verticalalignment='top', bbox=dict(boxstyle='round', facecolor='wheat',
        edgecolor='black', pad=0.5, alpha=0.8))

    fig.tight_layout()
    fig.savefig(f'{out_dir}/figure04_2D_pdf_slices.png', dpi=200, bbox_inches="tight")
    plt.close(fig)
    print(f'  Saved: {out_dir}/figure04_2D_pdf_slices.png')

# ----------------------------------------------------------------------
# Figure 5: Monte Carlo Convergence
# ----------------------------------------------------------------------
def plot_convergance(S0=100.0):
    """
    Show how the MC estimate converges as more paths are simulated.

    Page 473, Hull (MC Standard Error/Confidence intervals)
    """

    print("Computing convergence study...")
```

```
    V_exact  = bs_call(S0, K, r, sigma, T)
    N_range  = np.logspace(2, 6, 30).astype(int)
    V_estimates = np.zeros(len(N_range))
    V_errors = np.zeros(len(N_range))

    np.random.seed(11)
    for i, N in enumerate(N_range):
        V_estimates[i], V_errors[i] = mc_option_price(S0, K, r, sigma, T, N)

    fig, ax = plt.subplots(figsize=(9, 6))
    ax.semilogx(N_range, V_estimates, 'o-', color='tab:blue', markersize=5, linewidth
        =1.0, label='MC Estimate')
    ax.fill_between(N_range, V_estimates - 2*V_errors, V_estimates + 2*V_errors, alpha
        =0.2, color='tab:blue', label=r'$\pm 2 \sigma$ (95% CI)')
    ax.axhline(V_exact, color='tab:red', linestyle='--', linewidth=2.0, label=f'Exact (
        BS) = ${V_exact:.2f}')

    ax.set_xlabel('Number of Monte Carlo Paths, N', fontsize=12)
    ax.set_ylabel('Estimated Option Value, V ($)', fontsize=12)
    ax.set_title(fr'Monte Carlo Convergence ($S_0$ = \${S0:.0f})', fontsize=15)
    ax.legend(fontsize=12)
    ax.grid(True, alpha=0.3)

    fig.tight_layout()
    fig.savefig(f'{out_dir}/figure05_MC_convergence.png', dpi=200, bbox_inches="tight")
    plt.close(fig)
    print(f'  Saved: {out_dir}/figure05_MC_convergence.png')

# ---------------------------------------------------------------------
# Figure 6: PDF Heatmap
# ---------------------------------------------------------------------
def plot_pdf_heatmap(S0=100.0):
    """
    2D Heatmap: x = stock price, y = time, color = probability density.

    Page 84, Montgomerey, Runger, Hubele (Mean and Varince in Price Space)
    Page 315, Hull (Mean and Variance in log space)
    """

    print("Computing PDF heatmap...")
    t_grid = np.linspace(0.05, T, 500)
    S_grid = np.linspace(40, 250, 500)

    PDF = np.zeros((len(t_grid), len(S_grid)))
    for i, t_val in enumerate(t_grid):
        mu_ln    = np.log(S0) + (r - 0.5 * sigma**2) * t_val
        sigma_ln = sigma * np.sqrt(t_val)
        PDF[i, :] = (1.0 / (S_grid * sigma_ln * np.sqrt(2*np.pi)) * np.exp(-0.5 * ((np.
            log(S_grid) - mu_ln) / sigma_ln)**2))

    fig, ax = plt.subplots(figsize=(9, 6))
    S_mesh, t_mesh = np.meshgrid(S_grid, t_grid)
    pcm = ax.pcolormesh(S_mesh, t_mesh, PDF, cmap='inferno', shading='gouraud')

    S_expected = S0 * np.exp(r * t_grid)
    ax.plot(S_expected, t_grid, 'w--', linewidth=2.0, label=r'$E(S_T) = S_0 e^{rt}$')

    for sign, lbl in [(1, r'+1$\sigma$'), (-1, r'-1$\sigma$')]:
        mu_ln    = np.log(S0) + (r - 0.5 * sigma**2) * t_grid
        sigma_ln = sigma * np.sqrt(t_grid)
        S_bound = np.exp(mu_ln + sign * sigma_ln)
```

```
        ax.plot(S_bound, t_grid, 'w:', linewidth=1.0, label=lbl, alpha=0.7)

    ax.set_xlabel('Stock Price, S ($)', fontsize=12)
    ax.set_ylabel('Time (years)', fontsize=12)
    ax.set_title('PDF Evolution Heatmap\n' \
                 fr'($S_0$ = \${S0:.0f}, r = {r:.0%}, $\sigma$ = {sigma:.0%})', fontsize
                     =15)
    ax.legend(fontsize=12, loc='upper right', facecolor='black', edgecolor='white',
        labelcolor='white')
    fig.colorbar(pcm, ax=ax, label ='Probability Density')
    ax.set_xlim(40, 250)

    fig.tight_layout()
    fig.savefig(f'{out_dir}/figure06_pdf_heatmap.png', dpi=200, bbox_inches="tight")
    plt.close(fig)
    print(f'  Saved: {out_dir}/figure06_pdf_heatmap.png')

# ------------------------------------------------------------------------
# Main Driver
# ------------------------------------------------------------------------
def main():
    print("=" * 120)
    print("BSM Solver - Monte Carlo + Finite Difference + Analytical")
    print("=" * 120)
    print(fr'Parameters: K=${K}, r={r:.2%}, $\sigma$={sigma:.2%}, T={T} years')
    print(f'Monte Carlo: {N_paths:,} paths, {N_steps} time steps')
    print("=" * 120)

    np.random.seed(2026)

    S_arr, V_an, V_mc, V_err, F_fd = plot_option_value_curve()
    S_paths, t_arr          = plot_sample_paths()
    t_snap, S_centers, PDF_mc = plot_pdf_evolution()
    plot_pdf_slices()
    plot_convergance()
    plot_pdf_heatmap()

    print("\n" + "=" * 120)
    print("V(S) at selected stock prices")
    print("=" * 120)
    print(f'{'S ($)':>8} {'Analytical':>12} {'Monte Carlo':>12} {'Finite Difference'
        :>12} {'MC Error':>8}')
    print("-" * 120)

    S_fd_check, V_fd_check = fd_bsm(K, r, sigma, T, S_max=500.0, M=500, N_t=5000)
    check_prices = [60, 80, 100, 120, 140, 160]
    for S0 in check_prices:
        V_an_val      = bs_call(S0, K, r, sigma, T)
        V_mc_val, V_mc_err = mc_option_price(S0, K, r, sigma, T, N_paths)
        V_fd_vall     = np.interp(S0, S_fd_check, V_fd_check)
        print(f'{S0:>8.0f} {V_an_val:>12.2f} {V_mc_val:>12.2f} {V_fd_vall:>12.2f} {
            V_mc_err:>12.2f}')

    print('\nAll figures saved to:', out_dir)
    print('Done!')

if __name__ == "__main__":
    main()
```